Sprog og oversættere

Lexical analysis: From NFA to DFA; Minimizing a DFA

Part 1: From NFAs to DFAs

Hans Hüttel

(using content by Cosmin Oancea)

Converting a regular expression to an NFA

Last time we saw that one can convert a regular expression to a nondeterministic finite automaton.

But an NFA makes guesses – it is not a proper algorithm.

Can we get rid of the nondeterminism? If we can find an algorithm for that, then we can build a lexer from any regular expression.


We Can Do It!

A deterministic finite automaton

A deterministic finite automaton (DFA) is a very boring and predictable nondeterministic finite automaton.

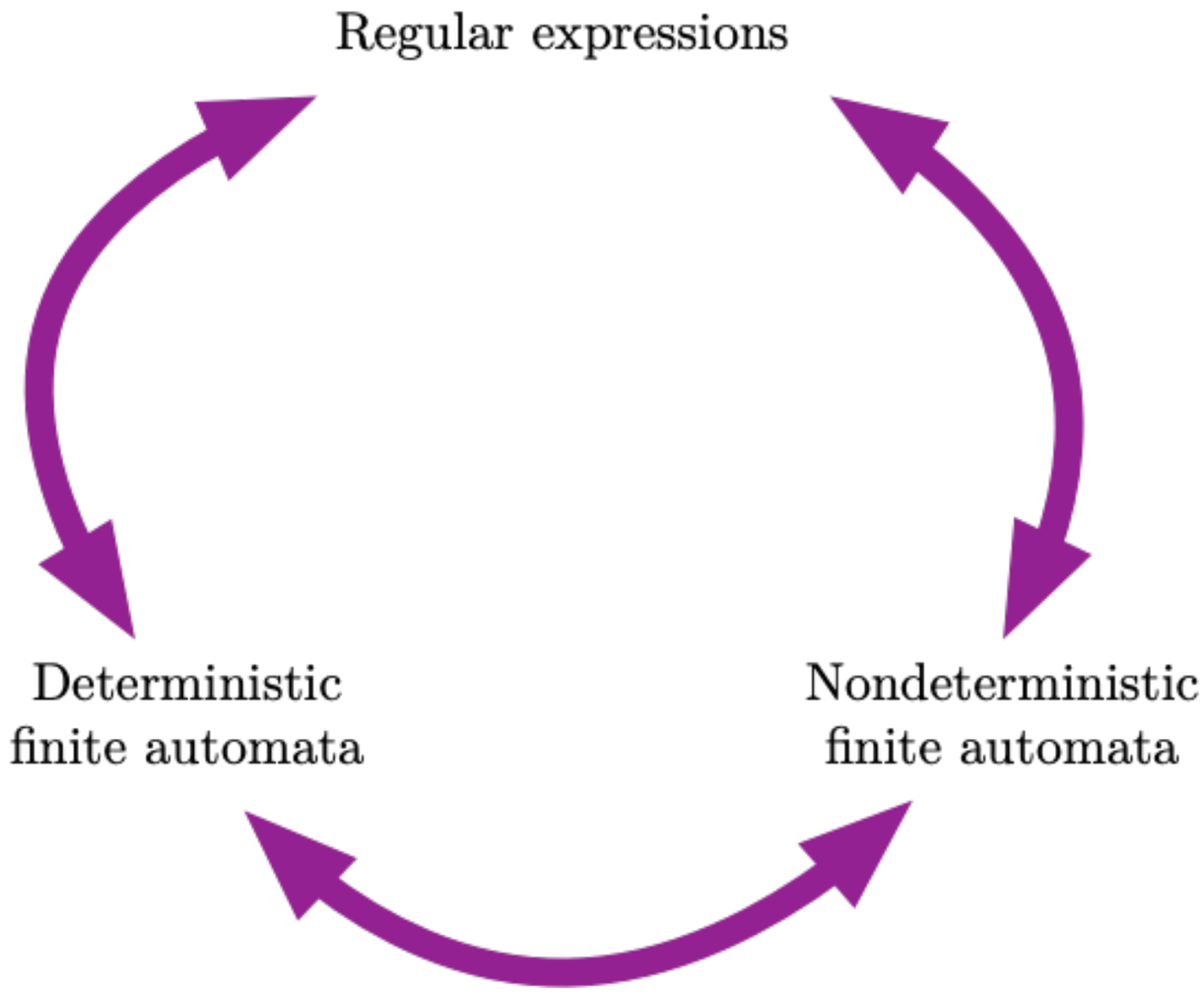
It satisfies the following two conditions:

- For every state q and character a, there is exactly one transition $q \xrightarrow{a} q'$ to some state q'.
- There are no ε-transitions at all.

(If you follow the course *Syntax and semantics*, have a look at Definition 1.5 in Sipser.)

The theorem behind it all

A result, known as Kleene's theorem, tells us that regular expressions have the same expressive power as two different machine models.



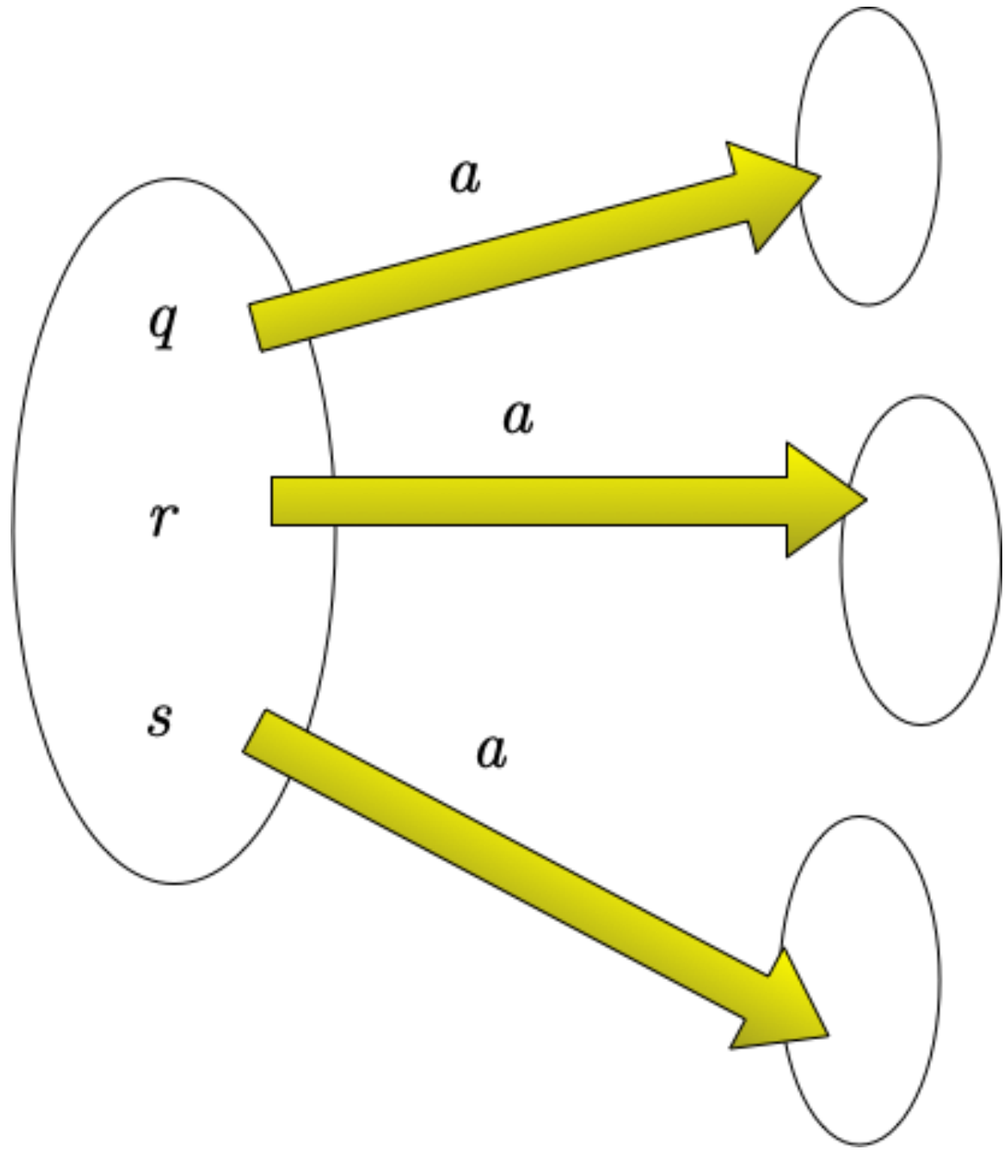
Converting an NFA to DFA: Idea

The main idea of converting an NFA N to a DFA M is to build an M whose states tell us *which states N can be in if it has read an input character.*

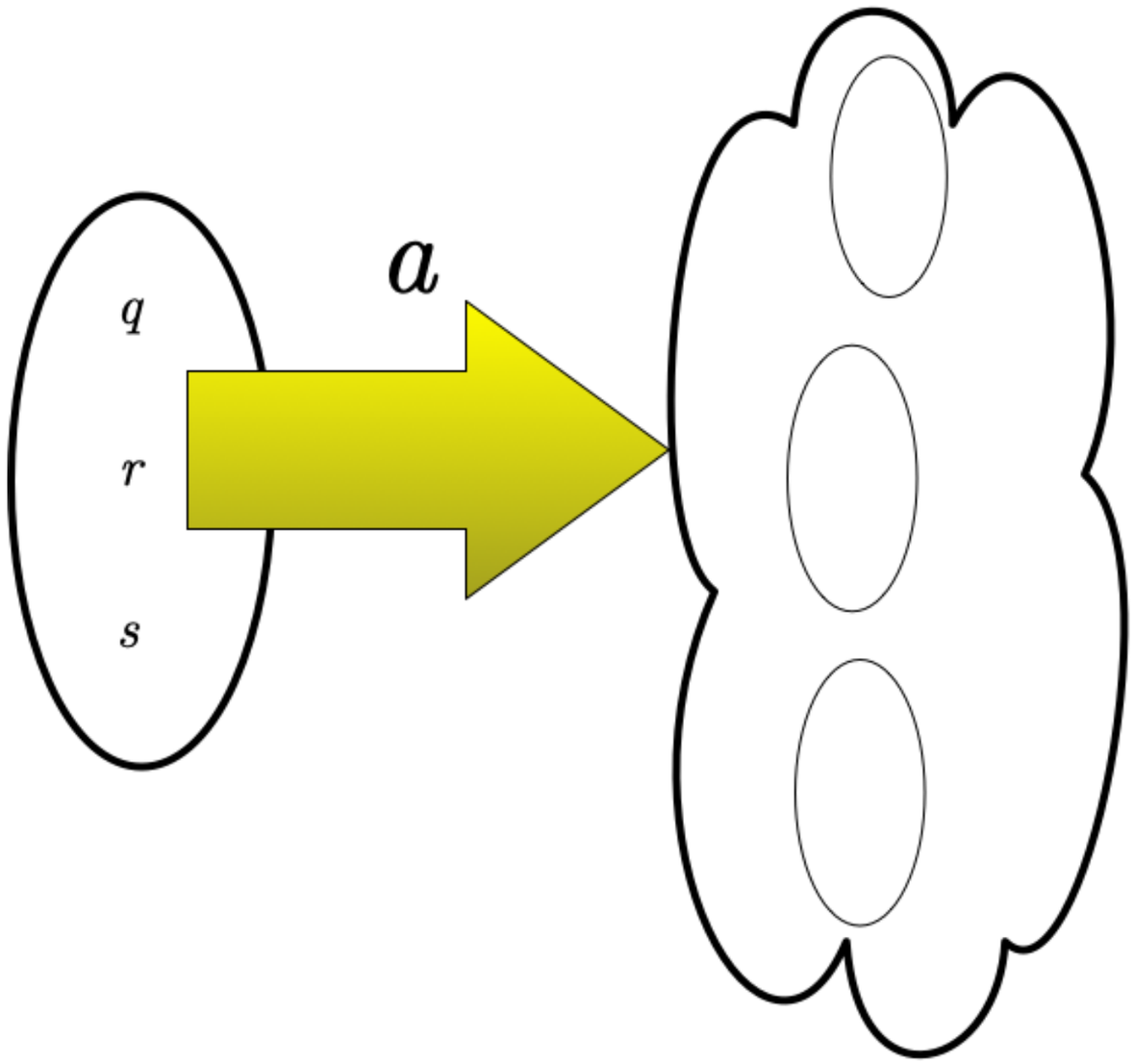
The states of our M will be sets of states from N.

For this reason we call the conversion algorithm *the subset construction.*

The general idea



The general idea



Building new transitions

In the new DFA, states are subsets of S, the state set of our NFA.

For any $A \subseteq S$ and character a, there is a transition $A \xrightarrow{a}_{\mathsf{DFA}} B$ where

$$B = \{q_2 \mid \exists q_1 \in A. q_1 \xrightarrow{a}_{\mathsf{NFA}} q_2\}$$

so B is the set of NFA states we could reach from A in the NFA using an a-transition.

The new start state

For a moment, let us assume that there are no ϵ-transitions. Then, if the start state in the NFA was s_0, the start state in the DFA will be

$$s_0^{\mathsf{DFA}} = \{s_0^{\mathsf{NFA}}\}$$

If there are ϵ-transitions, we need to be more careful.

The new final states

In an NFA, a string w is accepted if there a way to reach a final state s_f from s_0 by reading w.

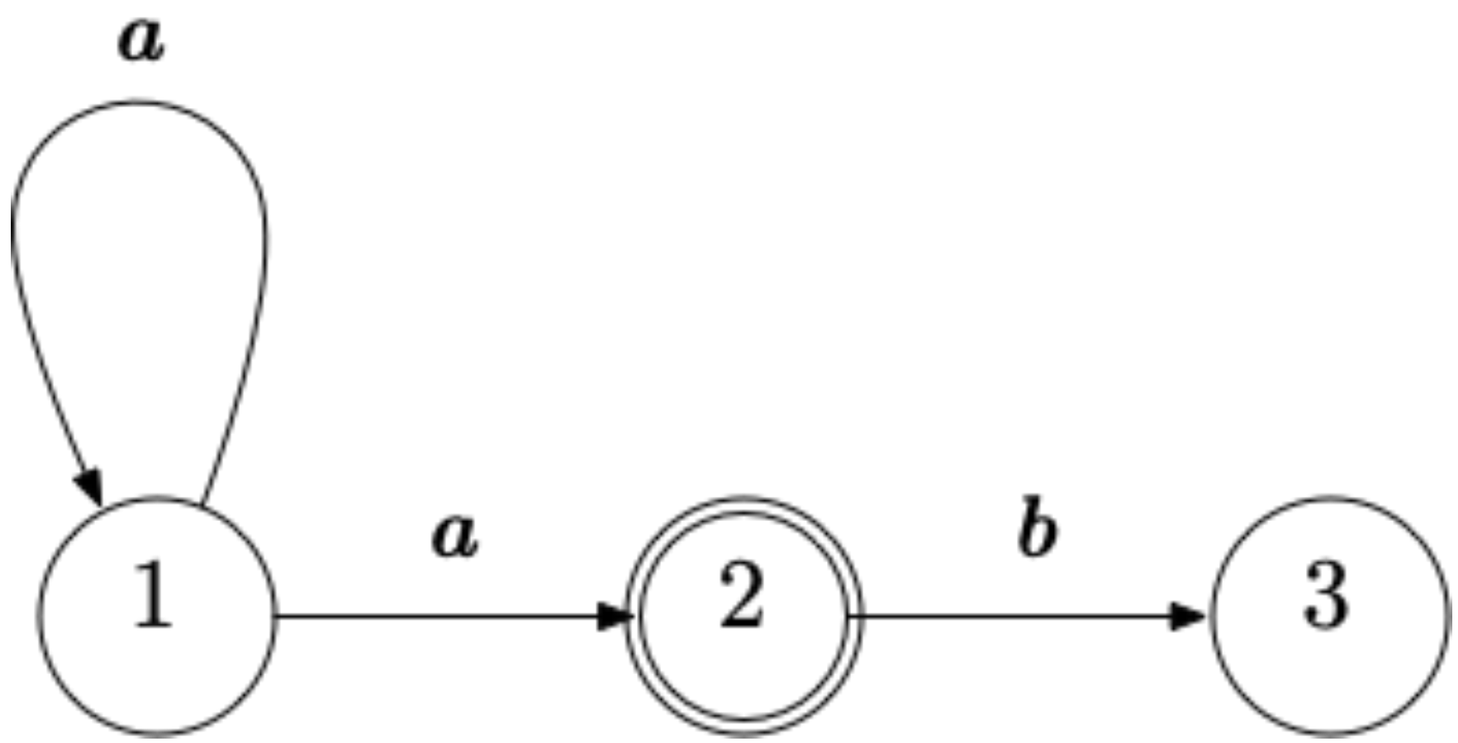
States in our new DFA record the sets of states that we can reach in the NFA.

So the new final states in the DFA that we build are the subsets S' of S that contain one or more final states from the NFA.

$$F_{\mathsf{DFA}} = \{S' \mid S' \subseteq S, S' \cap F_{\mathsf{NFA}} \neq \emptyset\}$$

A simple example

First let us consider an NFA that has no ε-transitions.



Why is this not a DFA?

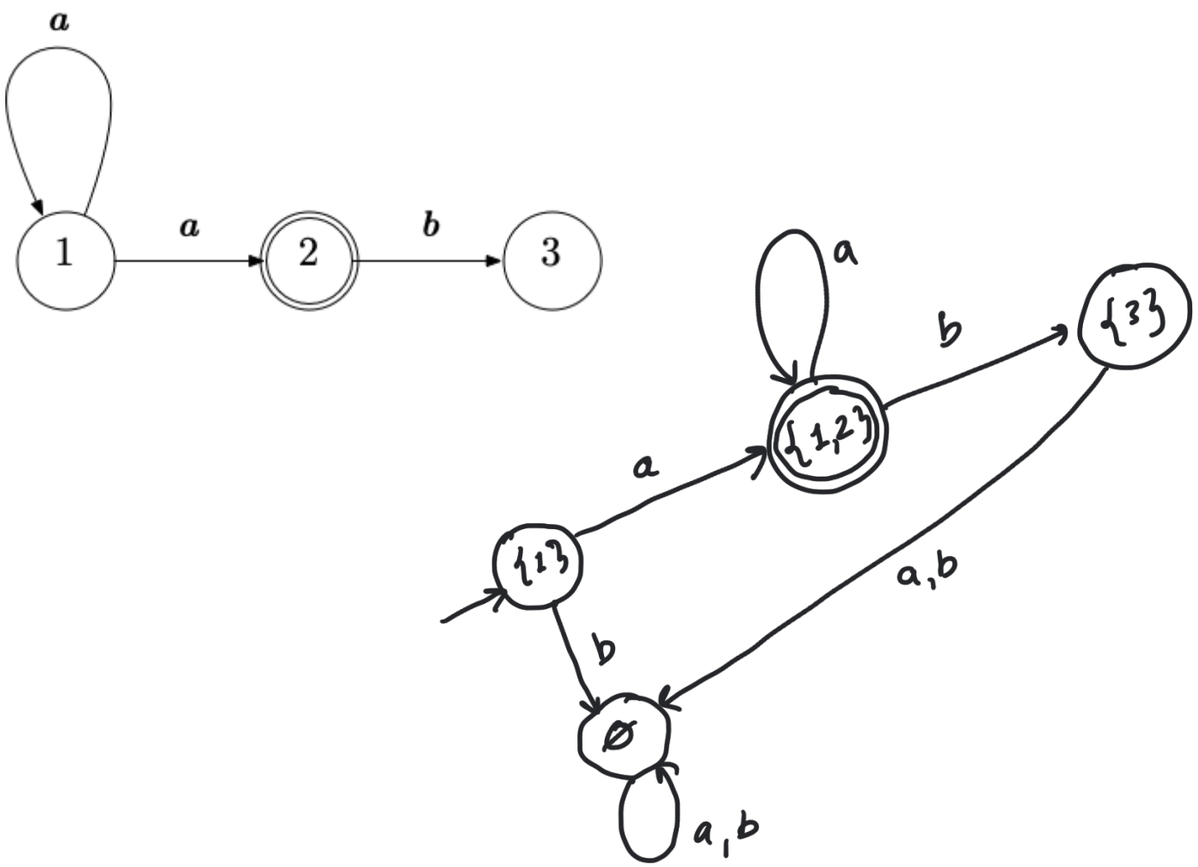
A simple example

There are two approaches, given a DFA with state set Q.

1. In Sipser, one builds a DFA whose set of states is $\mathcal{P}(Q)$, the family of all subsets of Q. If $|Q| = k$, this gives us 2^k states.
2. In Mogensen, one builds a DFA that only contains the reachable states. Sometimes, this will give us a smaller automaton.

We use the latter approach here.

A simple example



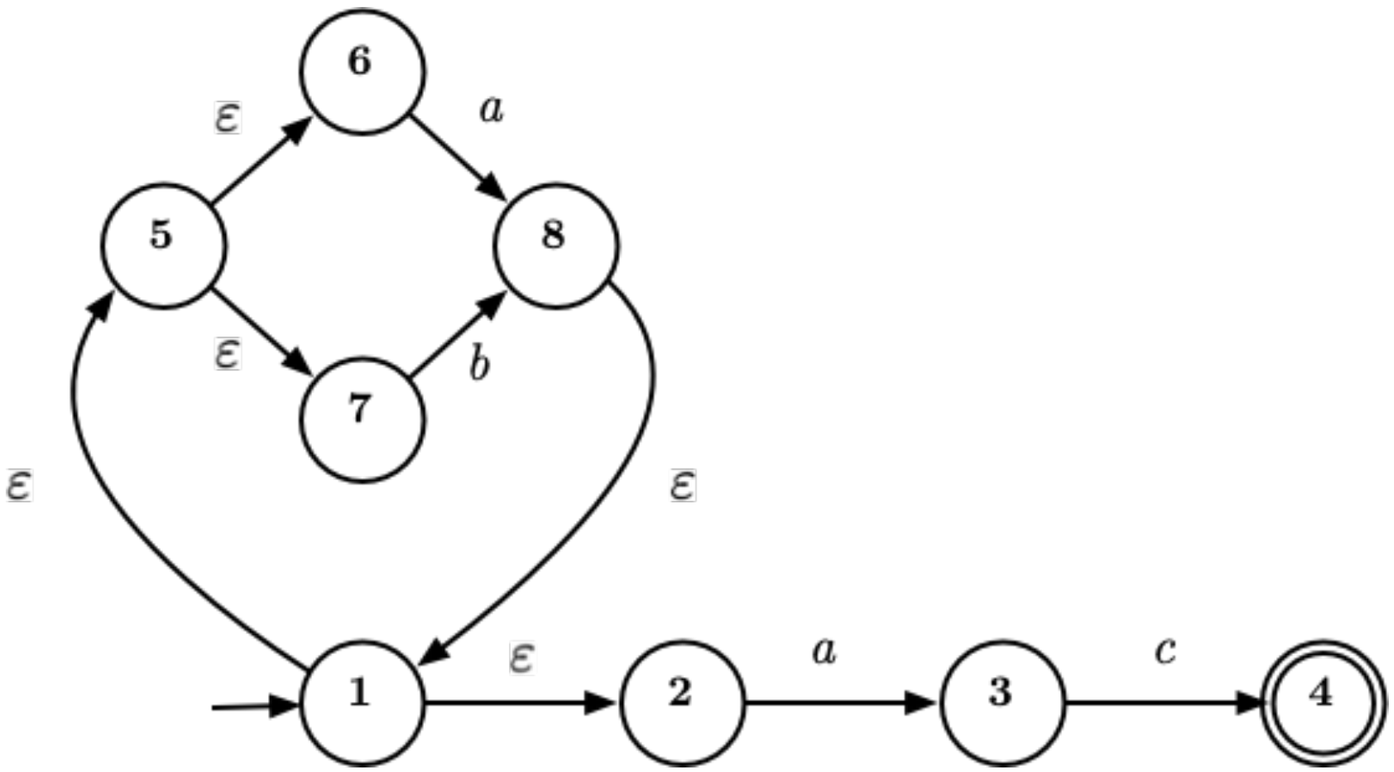
The epsilon closure

To get rid of ϵ-transitions, we need to find the set of all states in our NFA that we can reach from a given set of states M using only zero or more ϵ-transitions.

This set is called the *epsilon closure* of M and is denoted $\epsilon - \text{closure}(M)$

Finding the epsilon closure

It is often easy to see from the graph of an NFA what the epsilon closure of a set of states is.



It is easy to see that $\epsilon - \text{closure}(\{1\}) = \{1, 2, 5, 6, 7\}$. But how do we *compute* the epsilon-closure?

There is an algorithm for that!

The epsilon closure is defined by an equation

The states in the epsilon closure $\epsilon - \text{closure}(M)$ of M are

- the states in M
- along with every state s' that can be reached from $\epsilon - \text{closure}(M)$ with an ϵ-transition

so we have

$$\epsilon - \text{closure}(M) = M \cup \{s' \mid \exists s \in \epsilon - \text{closure}(M).s \xrightarrow{\epsilon} s'\}$$

The epsilon closure is defined by an equation

The states in the epsilon closure $\epsilon - \text{closure}(M)$ of M are

- the states in M
- along with every state s' that can be reached from $\epsilon - \text{closure}(M)$ with an ϵ-transition

so we have

$$\epsilon - \text{closure}(M) = M \cup \{s' \mid \exists s \in \epsilon - \text{closure}(M).s \xrightarrow{\epsilon} s'\}$$

In other words, $\epsilon - \text{closure}(M)$ is the set X such that

$$X = M \cup \{s' \mid \exists s \in X.s \xrightarrow{\epsilon} s'\}$$

This is a *set equation*. In a later part of this video, we see an algorithm for finding the solution to such an equation.

The subset construction – now with epsilons

The start state needs to be redefined now. If there are outgoing ϵ-transitions from s_0 in our NFA, we can start reading the input from any one of those states, too. This means that

$$s^{\mathsf{DFA}} = \epsilon - \text{closure}(s_0^{\mathsf{NFA}})$$

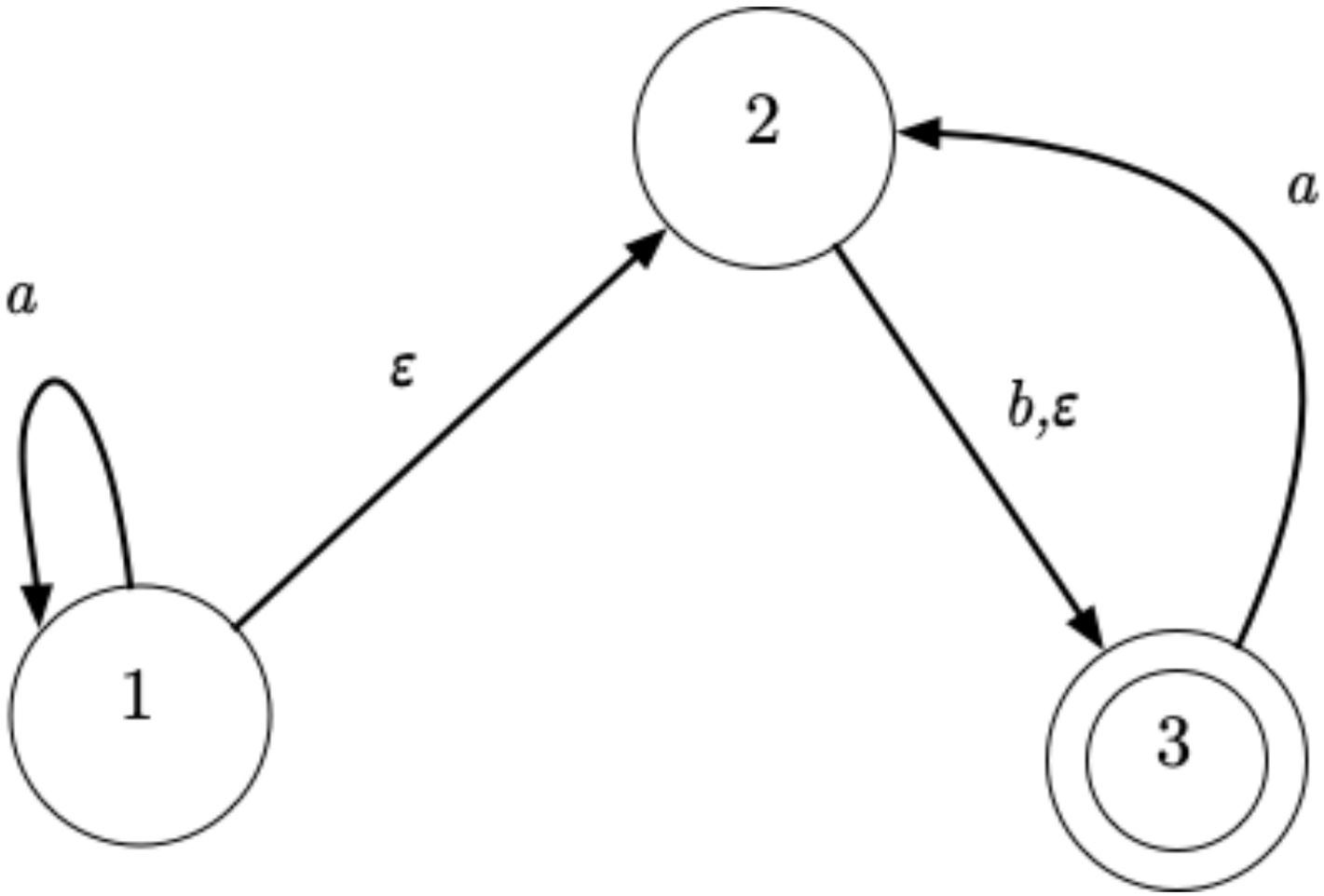
The subset construction – now with epsilons

The transition relation must also be redefined. For after we have read a symbol a, we can follow one or more ϵ-transitions.

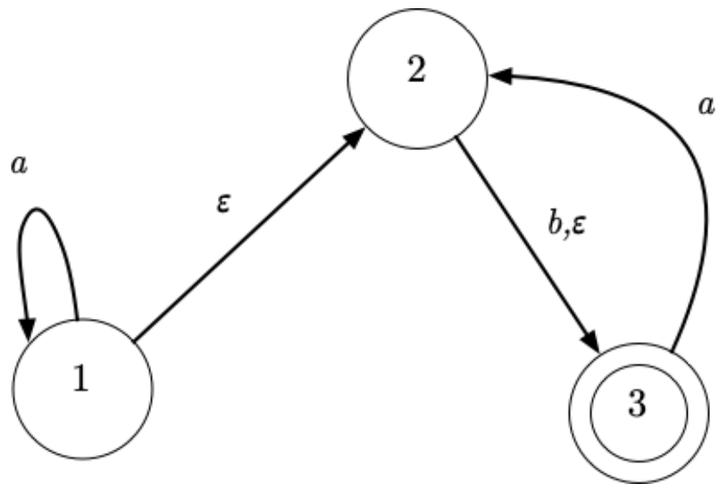
This means that for any $A \subseteq S$ and character a, there is a transition $A \xrightarrow{a}_{\mathsf{DFA}} B$ where

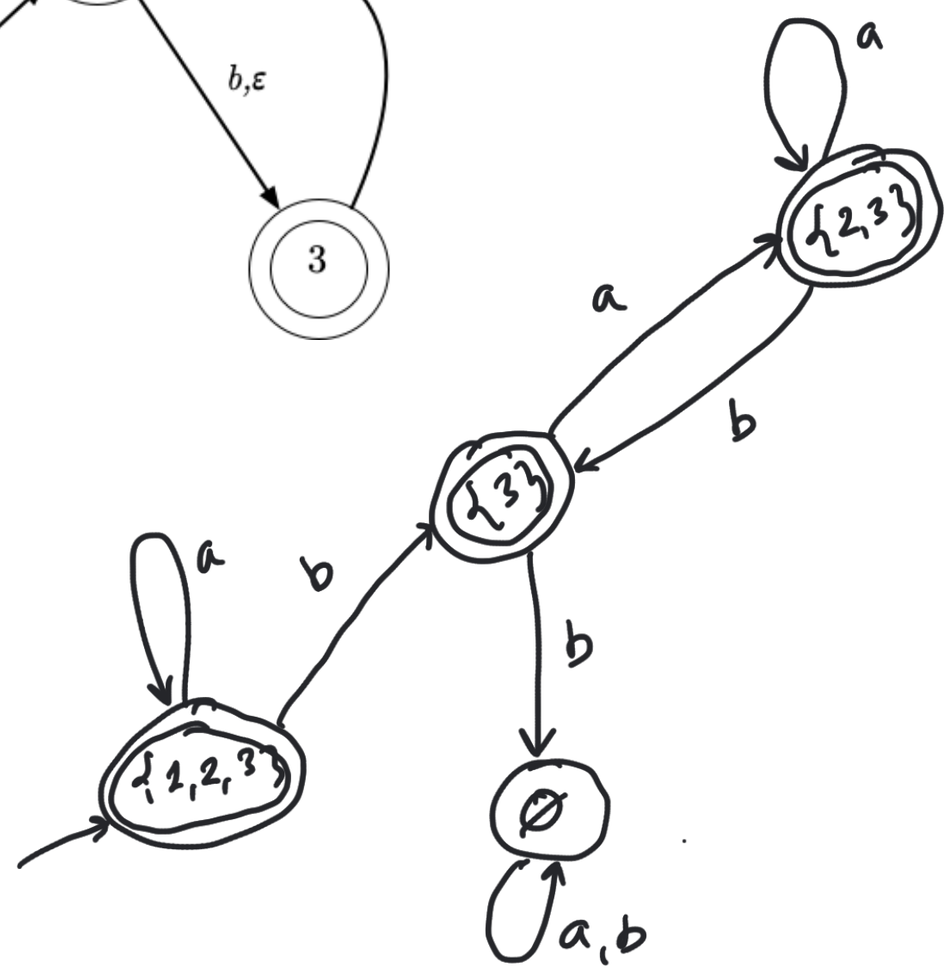
$$B = \epsilon - \text{closure}(\{q_2 \mid \exists q_1 \in A . q_1 \xrightarrow{a}_{\mathsf{NFA}} q_2\})$$

The subset construction – now with epsilons



The subset construction – now with epsilons





Summing up

Given an NFA $N = (S, \Sigma, s_0, \{\xrightarrow{a} | \ a \in \Sigma\}, F)$ we can build a DFA $D = (S', \Sigma, s_0', \{\xrightarrow{a} | \ a \in \Sigma\}, F')$ where

- $S' = \mathcal{P}(S)$
- $s_0' = \epsilon - \text{closure}(\{s_0\})$
- The transition relation is for every $A \in S'$ given by

$$A \xrightarrow{a} B \text{ if } B = \epsilon - \text{closure}(\{q_2 \mid \exists q_1 \in A . q_1 \xrightarrow{a} q_2\})$$

- $F' = \{M \subseteq S \mid M \cap F \neq \emptyset\}$

and where it holds that $L(D) = L(N)$.

Is it inefficient to build a DFA from an NFA?

We know from set theory that if a set[1] S has size n, the power set[2] has 2^n elements.

- The size of the DFA may reach (rarely) 2^n for a n-state NFA.
- DFA can be run in $k \cdot |v|$, with k small ct, and $|v|$ length of input,
- NFA can be run in $c \cdot |n| \cdot |v|$, where $c > k$ constant, $|n|$ size of NFA, $|v|$ length of input

[1] Dansk: mængde
[2] Dansk: potensmængde

Is it inefficient to build a DFA from an NFA?

We know from set theory that if a set[1] S has size n, the power set[2] has 2^n elements.

- The size of the DFA may reach (rarely) 2^n for a n-state NFA.
- DFA can be run in $k \cdot |v|$, with k small ct, and $|v|$ length of input,
- NFA can be run in $c \cdot |n| \cdot |v|$, where $c > k$ constant, $|n|$ size of NFA, $|v|$ length of input
- DFA much faster $\Rightarrow$ only when size of resulting DFA is too big consider using an NFA directly.

[1] Dansk: mængde
[2] Dansk: potensmængde